

Go Concurrency & SQL — Cheat Sheet

Паттерны каналов · sync-примитивы · SQL задачи

Каналы: основы

Небуферизованный — синхронный (блок до получателя). Буферизованный — асинхронный до заполнения.

```
ch := make(chan int)      // sync
ch := make(chan int, 10)  // buffered
```

Аксиомы:

- send в nil/closed → **panic**
- recv из closed → zero + false
- recv/send в nil → блок навсегда
- select: nil-ветка никогда не выбирается

Pipeline

Цепочка горутин: каждая читает in, пишет в out, закрывает после себя.

```
func stage(in <-chan int) <-chan int {
    out := make(chan int)
    go func() {
        defer close(out)
        for v := range in { out <- v*v }
    }()
    return out
}
// usage: src -> stage -> stage -> sink
```

Fan-out / Fan-in

Fan-out: N воркеров читают один канал.

Fan-in: слить N каналов в один.

```
func merge(cs ...<-chan int) <-chan int {
    var wg sync.WaitGroup
    out := make(chan int)
    wg.Add(len(cs))
    for _, c := range cs {
        go func(ch <-chan int) {
            defer wg.Done()
            for v := range ch { out <- v }
        }(c)
    }
    go func() { wg.Wait(); close(out) }()
    return out
}
```

Done-channel / Отмена

```
done := make(chan struct{})
defer close(done) // broadcast сигнал-

go func() {
    select {
    case out <- process():
    case <-done: return // leak-safe
    }
```

```
}()
```

Предпочти context.Context в production-коде.

Timeout / Select

```
select {
case v := <-ch:
    handle(v)
case <-time.After(2 * time.Second):
    // timeout
case <-ctx.Done():
    return ctx.Err()
}
```

Семафор (ограничение пула)

```
sem := make(chan struct{}, N)
for _, job := range jobs {
    sem <- struct{}{} // acquire slot
    go func(j Job) {
        defer func() { <-sem }()
        process(j)
    }(job)
}
```

Mutex / RWMutex

```
var mu sync.RWMutex
// чтение параллельно():
mu.RLock(); v := m[k]; mu.RUnlock()
// запись эксклюзивно():
mu.Lock(); defer mu.Unlock()
m[k] = v
```

- defer mu.Unlock() сразу после Lock()
- Pointer receiver — мьютекс **нельзя копировать**
- **Не реентерабелен** — повторный Lock = дедлок

WaitGroup / Once / Atomic

```
var wg sync.WaitGroup
wg.Add(1) // до go func
go func() {
    defer wg.Done(); work()
}()
wg.Wait()

var once sync.Once
once.Do(initDB) // ровно 1 раз

var cnt int64
atomic.AddInt64(&cnt, 1)
v := atomic.LoadInt64(&cnt)
```

Типичные ошибки

- **Goroutine leak:** горутина заблокирована навсегда → done-ch или ctx
- **Data race:** r/w без sync → go test -race
- **wg.Add внутри go func** → Wait может выйти раньше

- **Value-receiver + Mutex** → копирует мьютекс
- **Закрывает получатель** → panic у второго отправителя

SQL: схема

```
employees(id, name, dept_id, salary)
departments(id, name)
orders(id, cust_id, amount, created_at)
order_items(id, order_id, product_id, qty)
products(id, name, price)
```

SQL задачи

1. Средняя ЗП по отделам, только > 80k

```
SELECT d.name, AVG(e.salary) avg_sal
FROM employees e
JOIN departments d ON d.id = e.dept_id
GROUP BY d.name
HAVING AVG(e.salary) > 80000
ORDER BY avg_sal DESC;
```

2. Топ-3 клиента по сумме заказов

```
SELECT cust_id, SUM(amount) total
FROM orders
GROUP BY cust_id
ORDER BY total DESC LIMIT 3;
```

3. Сотрудники без отдела (LEFT JOIN)

```
SELECT e.name FROM employees e
LEFT JOIN departments d ON d.id = e.dept_id
WHERE d.id IS NULL;
```

4. Выручка за 7 дней по дням

```
SELECT DATE(created_at) day,
       SUM(amount) revenue
FROM orders
WHERE created_at >= NOW()-INTERVAL '7 days'
GROUP BY day ORDER BY day;
```

5. Ранг ЗП внутри отдела (window)

```
SELECT name, salary,
       RANK() OVER (
           PARTITION BY dept_id
           ORDER BY salary DESC
       ) rn
FROM employees;
```

6. Товары без заказов

```
-- NOT IN опасен при NULL; лучше:
SELECT p.id, p.name FROM products p
LEFT JOIN order_items oi
ON oi.product_id = p.id
WHERE oi.id IS NULL;
```

Правила: wg.Add до go · канал закрывает отправитель · Mutex нереентерабелен · -race обязателен · HAVING фильтрует агрегаты · оконные функции не убирают строки